

CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

September 30, 2025

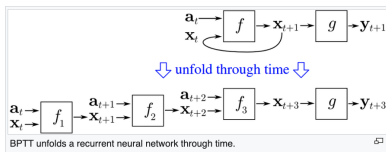
Lecture 11

How does the Backpropagation through time work in RNN?

Backpropagation through time (BPTT) is a gradient-based technique for training certain types of recurrent neural networks. It can be used to train Elman networks. The algorithm was independently derived by numerous researchers.^{[1][2][3]}

Algorithm [\[edit \]](#)

The training data for a recurrent neural network is an ordered sequence of k input-output pairs,

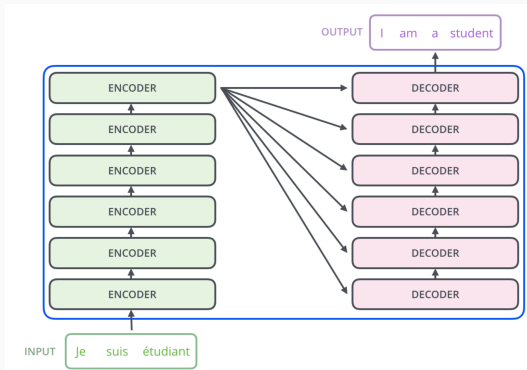


$\langle \mathbf{a}_0, \mathbf{y}_0 \rangle, \langle \mathbf{a}_1, \mathbf{y}_1 \rangle, \langle \mathbf{a}_2, \mathbf{y}_2 \rangle, \dots, \langle \mathbf{a}_{k-1}, \mathbf{y}_{k-1} \rangle$. An initial value must be specified for the hidden state $\mathbf{x}_0$. Typically, a vector of all zeros is used for this purpose.

BPTT begins by unfolding a recurrent neural network in time. The unfolded network contains k inputs and outputs, but every copy of the network shares the same parameters. Then the [backpropagation](#) algorithm is used to find the gradient of the cost with respect to all the network parameters.

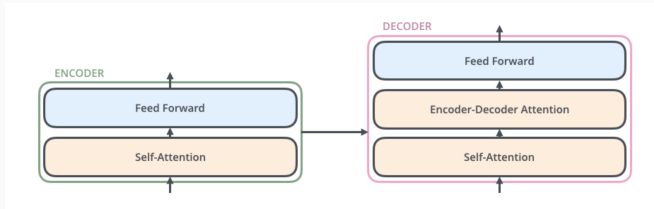
Transformer

- An architecture that relies **entirely on attention** without using CNN/RNN
- Proposed in “Attention Is All You Need” (Vaswani et al., 2017)
- Initially used for neural machine translation



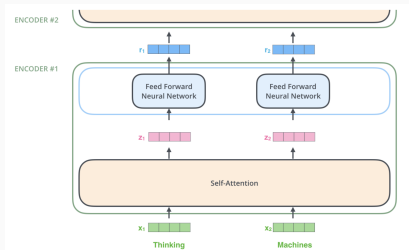
Encoder and Decoder

- Self attention layer: the main architecture used in Transformer
- Decoder: will have another attention layer to help it focuses on relevant parts of input sentences.



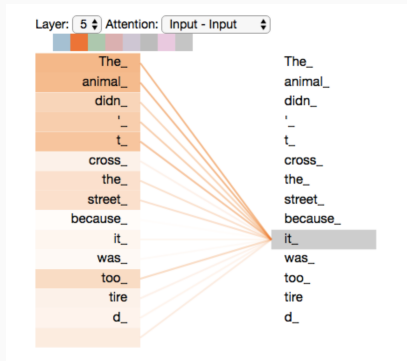
Encoder

- Each word has a corresponding “latent vector” (initially the word embedding for each word)
- Each layer of encoder:
 - Receive a list of vectors as input
 - Passing these vectors to a **self-attention** layer
 - Then passing them into a feed-forward layer
 - Output a list of vectors



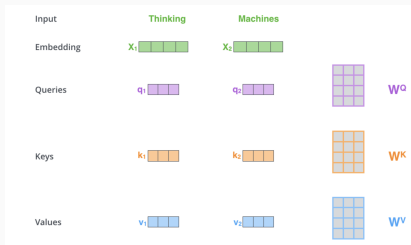
Self-attention layer

- Main idea: The actual meaning of each word may be related to other words in the sentence
- The actual meaning (latent vector) of each word is a weighted (attention) combination of other words (latent vectors) in the sentences



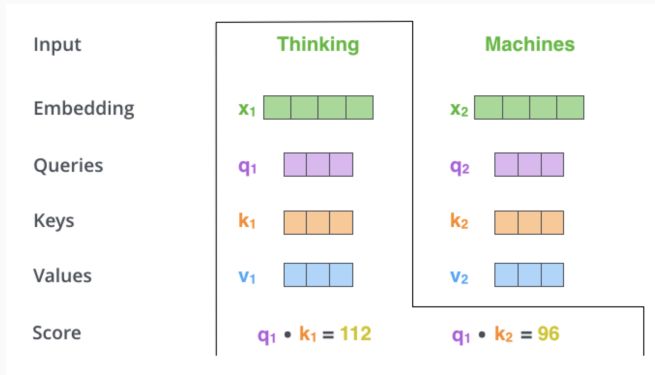
Self-attention layer

- Input latent vectors: $\mathbf{x}_1, \dots, \mathbf{x}_n$
- Self-attention parameters: W^Q, W^K, W^V (weights for query, key, value)
- For each word i , compute
 - Query vector: $q_i = \mathbf{x}_i W^Q$
 - Key vector: $k_i = \mathbf{x}_i W^K$
 - Value vector: $v_i = \mathbf{x}_i W^V$



Self-attention layer

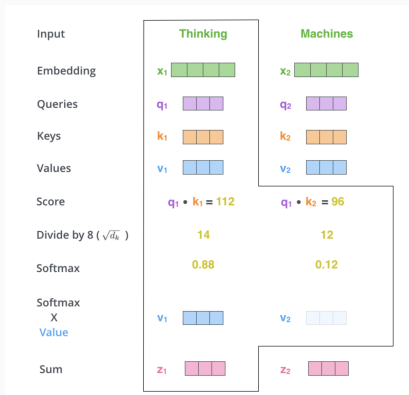
- For each word i , compute the scores to determine how much focus to place on other input words
 - The **attention** score for word j to word i : $q_i^T k_j$



Self-attention layer

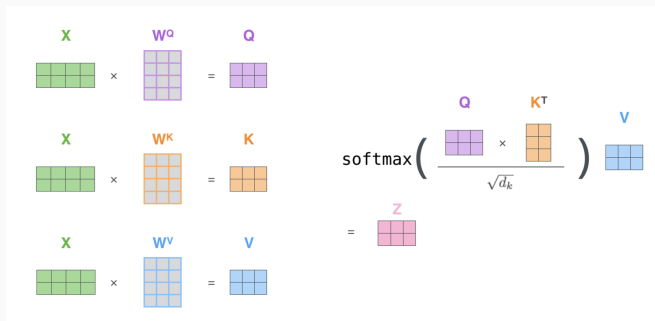
- For each word i , the output vector

$$\sum_j s_{ij} v_j, \quad s_i = \text{softmax}(q_i^T k_1, \dots, q_i^T k_n)$$



Matrix form

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad Z = \text{softmax}(QK^T)V$$

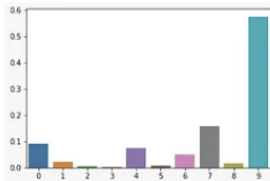


Softmax Temperature

- $\frac{\exp(z_i / T)}{\sum \exp(z_i / T)}$
- <https://medium.com/mlearning-ai/softmax-temperature-5492e4007f71>

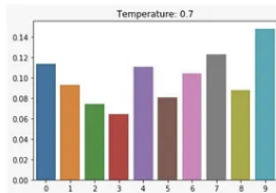
SOFTMAX WITHOUT TEMPERATURE ($T=1$)

$$\frac{e^{z_i}}{\sum_j e^{z_j}}$$



SOFTMAX WITH TEMPERATURE

$$\frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$



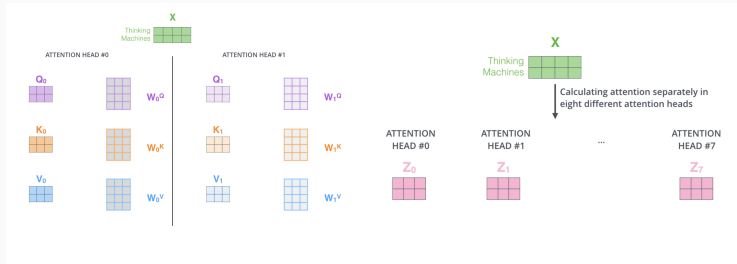
LESS ENTROPY

INCREASE IN ENTROPY
WITH INCREASE IN T

MORE ENTROPY

Multiple heads

- Multi-headed attention: use multiple set of (*key*, *value*, *query*) weights
- Each head will output a vector Z_i



Multiply with weight matrix to reshape

- Gather all the outputs $Z_1, \dots, Z_k$
- Multiply with a weight matrix to reshape
- Then pass to the next fully connected layer

1) Concatenate all the attention heads



2) Multiply with a weight matrix W^O that was trained jointly with the model

X



3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



Overall architecture

1) This is our input sentence*

Thinking
Machines

2) We embed each word*



3) Split into 8 heads.
We multiply X or R with weight matrices



4) Calculate attention using the resulting $Q/K/V$ matrices



5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

...



W^O

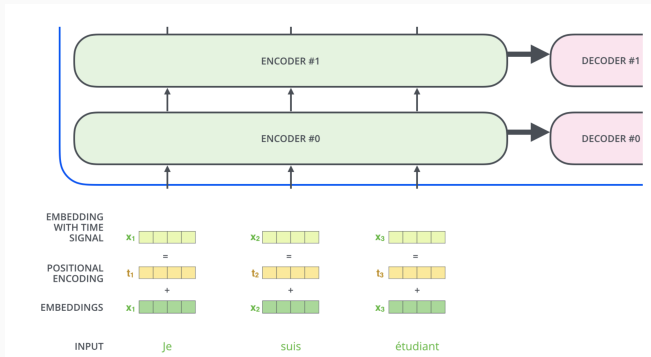


Z



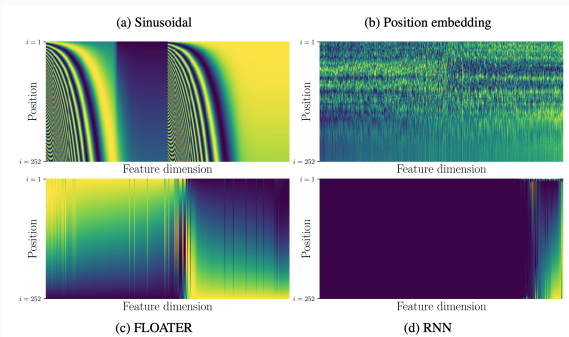
Position encoding

- The above architecture ignores the sequential information
- Add a position encoding vector to each x_i (according to i)



Types of position encoding

- Sin/cosine functions with different wavelengths (used in the original Transformer)
 - smooth, parameter-free
- Position embedding: learn a latent vector for each position
 - non-smooth, data-driven (learnable)
- Neural ODE embedding (Liu et al., 2020)
 - smooth, data-driven (learnable)



Relative Position Encoding

- Inject position embedding based on the relative position in attention

$$\alpha_{ij} = (\mathbf{x}_i W^Q)(\mathbf{x}_j W^K + \mathbf{p}_{\text{clip}(i-j, k)}^K).$$

- Each $\mathbf{p}_r^K, r = 1, \dots, k$ is a learnable vector.
- The maximum relative position is clipped to a maximum absolute value of k . It is hypothesized that precise relative position information is not useful beyond a certain distance. Clipping the maximum distance also enables the model to generalize to sequence lengths not seen during training.
- Various other ways to add relative position encoding

TL;DR: You have to play around with model.py yourself

minGPT / mingpt / model.py



mishig25 Use XOR operator ^ for checking assertion `type\_given XOR params\_gi...`

Code

Blame

310 lines (272 loc) · 14.3 KB

```
1  """
2  Full definition of a GPT Language Model, all of it in this single file.
3
4  References:
5  1) the official GPT-2 TensorFlow implementation released by OpenAI:
6  https://github.com/openai/gpt-2/blob/master/src/model.py
7  2) huggingface/transformers PyTorch implementation:
8  https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py
9  """
10
11  import math
12
13  import torch
14  import torch.nn as nn
15  from torch.nn import functional as F
16
17  from mingpt.utils import CfgNode as CN
18
19  # -----
20
21  class NewGELU(nn.Module):
22      """
23      Implementation of the GELU activation function currently in Google BERT repo (identical to OpenAI GPT).
24      Reference: Gaussian Error Linear Units (GELU) paper: https://arxiv.org/abs/1606.08415
25      """
26      def forward(self, x):
27          return 0.5 * x * (1.0 + torch.tanh(math.sqrt(2.0 / math.pi) * (x + 0.044715 * torch.pow(x, 3.0))))
28
29  class CausalSelfAttention(nn.Module):
30      """
```

Print out things line by line

```
[ ] device = 'cpu'
    prompt = "Binghamton University, the"
    tokenizer = BPETokenizer()
    x = tokenizer(prompt).to(device)
```

downloading <https://openaiublic.blob.core.windows.net/gpt-2/models/124M/encoder.json> to /root/.cache/mingpt/encoder.json
downloading <https://openaiublic.blob.core.windows.net/gpt-2/models/124M/vocab.bpe> to /root/.cache/mingpt/vocab.bpe

```
[ ] y = model.generate(x, max_new_tokens=40, do_sample=False, top_k=40)
```

```
idx = x
b, t = idx.size()
assert t <= model.block_size, f"Cannot forward sequence of length {t}, block size is only {model.block_size}"
pos = torch.arange(0, t, dtype=torch.long, device=device).unsqueeze(0) # shape (1, t)

# forward the GPT model itself
tok_emb = model.transformer.wte(idx) # token embeddings of shape (b, t, n_embd)
pos_emb = model.transformer.wpe(pos) # position embeddings of shape (1, t, n_embd)
x = model.transformer.drop(tok_emb + pos_emb)

print(x, x.shape)
stop
for block in self.transformer.h:
    x = block(x)
x = self.transformer.ln_f(x)
logits = self.lm_head(x)

# if we are given some desired targets also calculate the loss
loss = None
if targets is not None:
    loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), ignore_index=-1)
```

generate

```
[ ] device = 'cpu'
    prompt = "Binghamton University, the"
    tokenizer = BPETokenizer()
    x = tokenizer(prompt).to(device)
```

downloading <https://openaiublic.blob.core.windows.net/gpt-2/models/124M/encoder.json> to /root/.cache/mingpt/encoder.json
downloading <https://openaiublic.blob.core.windows.net/gpt-2/models/124M/vocab.bpe> to /root/.cache/mingpt/vocab.bpe

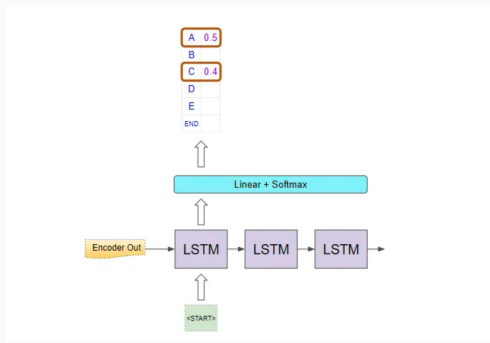
```
[ ] y = model.generate(x, max_new_tokens=40, do_sample=False, top_k=40)
```

generate: which line is the main inference of the model?

```
def generate(self, idx, max_new_tokens, temperature=1.0, do_sample=False, top_k=None):
    """
    Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete
    the sequence max_new_tokens times, feeding the predictions back into the model each time.
    Most likely you'll want to make sure to be in model.eval() mode of operation for this.
    """
    for _ in range(max_new_tokens):
        # if the sequence context is growing too long we must crop it at block_size
        idx_cond = idx if idx.size(1) <= self.block_size else idx[:, -self.block_size:]
        # forward the model to get the logits for the index in the sequence
        logits, _ = self(idx_cond)
        # pluck the logits at the final step and scale by desired temperature
        logits = logits[:, -1, :] / temperature
        # optionally crop the logits to only the top k options
        if top_k is not None:
            v, _ = torch.topk(logits, top_k)
            logits[logits < v[:, [-1]]] = -float('Inf')
        # apply softmax to convert logits to (normalized) probabilities
        probs = F.softmax(logits, dim=-1)
        # either sample from the distribution or take the most likely element
        if do_sample:
            idx_next = torch.multinomial(probs, num_samples=1)
        else:
            _, idx_next = torch.topk(probs, k=1, dim=-1)
        # append sampled index to the running sequence and continue
        idx = torch.cat((idx, idx_next), dim=1)

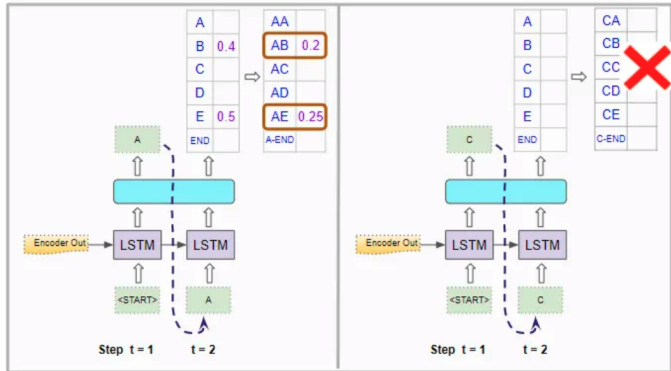
    return idx
```

Beam Search

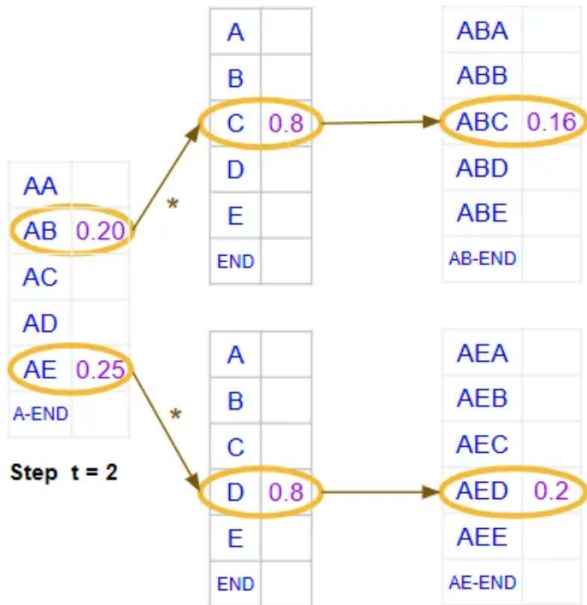


- <https://towardsdatascience.com/foundations-of-nlp-explained-visually-beam-search-how-it-works-1586b9849a24>
- https://d2l.ai/chapter_recurrent-modern/beam-search.html

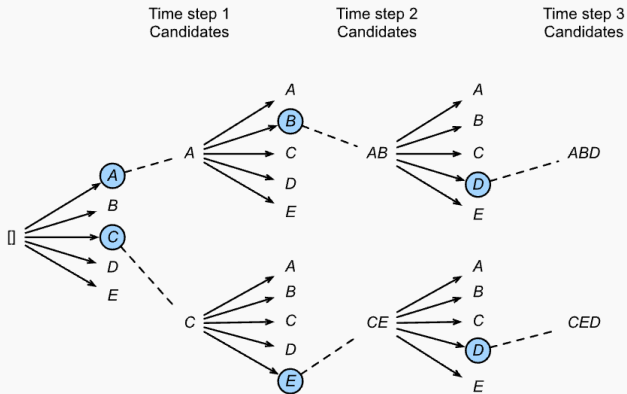
Beam Search



Beam Search



Beam Search



generate: What's max new tokens?

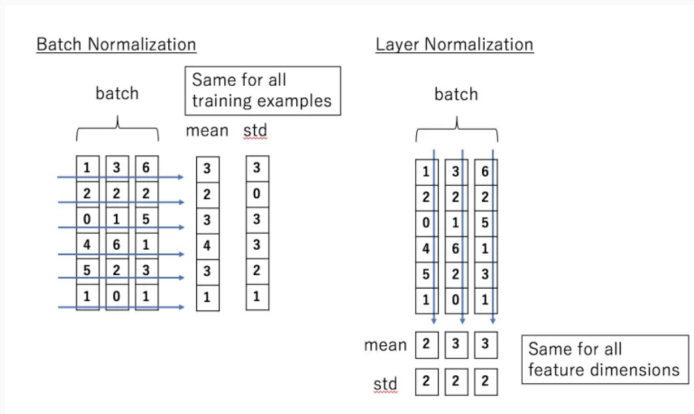
```
def generate(self, idx, max_new_tokens, temperature=1.0, do_sample=False, top_k=None):
    """
    Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete
    the sequence max_new_tokens times, feeding the predictions back into the model each time.
    Most likely you'll want to make sure to be in model.eval() mode of operation for this.
    """
    for _ in range(max_new_tokens):
        # if the sequence context is growing too long we must crop it at block_size
        idx_cond = idx if idx.size(1) <= self.block_size else idx[:, -self.block_size:]
        # forward the model to get the logits for the index in the sequence
        logits, _ = self(idx_cond)
        # pluck the logits at the final step and scale by desired temperature
        logits = logits[:, -1, :] / temperature
        # optionally crop the logits to only the top k options
        if top_k is not None:
            v, _ = torch.topk(logits, top_k)
            logits[logits < v[:, [-1]]] = -float('Inf')
        # apply softmax to convert logits to (normalized) probabilities
        probs = F.softmax(logits, dim=-1)
        # either sample from the distribution or take the most likely element
        if do_sample:
            idx_next = torch.multinomial(probs, num_samples=1)
        else:
            _, idx_next = torch.topk(probs, k=1, dim=-1)
        # append sampled index to the running sequence and continue
        idx = torch.cat((idx, idx_next), dim=1)

    return idx
```

```
self.transformer = nn.ModuleDict(dict(  
    wte = nn.Embedding(config.vocab_size, config.n_embd),  
    wpe = nn.Embedding(config.block_size, config.n_embd),  
    drop = nn.Dropout(config.embd_pdrop),  
    h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),  
    ln_f = nn.LayerNorm(config.n_embd),  
))
```

1. As the depth of the network increases, the distribution of feature values in each layer will gradually approach the upper and lower ends of the activation function's output range, causing the activation function to saturate. Continuing in this way can lead to gradient vanishing. Normalization can recenter the distribution of feature values to a standard normal distribution, ensuring that the feature values fall within the range where the activation function is more sensitive to inputs.
2. We assume Independent and Identically Distributed (IID), which assumes that the training data and test data follow the same distribution. Normalizing the training and test data can prevent the influence of different data distributions on the model.

LayerNorm BatchNorm



https://medium.com/@florian_algo/batchnorm-and-layernorm-2637f46a998b

LayerNorm BatchNorm

```
CLASS torch.nn.LayerNorm(normalized_shape, eps=1e-05, elementwise_affine=True, bias=True, device=None,
dtype=None) [SOURCE]
```

Applies Layer Normalization over a mini-batch of inputs.

This layer implements the operation as described in the paper [Layer Normalization](#)

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

The mean and standard-deviation are calculated over the last D dimensions, where D is the dimension of `normalized_shape`. For example, if `normalized_shape` is `(3, 5)` (a 2-dimensional shape), the mean and standard-deviation are computed over the last 2 dimensions of the input (i.e. `input.mean((-2, -1))`). γ and β are learnable affine transform parameters of `normalized_shape` if `elementwise_affine` is `True`. The standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`.